

Competitive Programming

Section 38: Static Queries and Binary Lifting

Competitive Programming Course

2026-08-29

Table of contents

1 Sparse tables for static range queries	3
2 Sparse table worked query	5
2.1 Complete sparse table	5
3 Binary lifting: precompute long parent jumps	7
3.1 The problem with following parents one by one	7
3.2 What one table entry means	7
3.3 Building longer jumps from shorter jumps	8
3.4 A complete table on a small tree	8
4 Using the table to answer a query	10
4.1 Why the query is logarithmic	10
5 Complete implementation	11
6 Binary lifting correctness summary	13
7 Recognition	14
8 Local exercises	15
9 How to use the checker	16
10 Static means a different tradeoff	17
10.1 Second sparse-table trace	17
11 Notes, mistakes, and formative checks	18
12 Submission workflow	19

1 Sparse tables for static range queries

The first question is not which data structure you remember. It is whether the data can change. If there are no updates, preprocessing never becomes stale, so we can prepare answers for many useful pieces in advance.

Compare the main choices:

Situation	Natural tool	Query time
static range sum	prefix sums	$O(1)$
static range minimum or GCD	sparse table	$O(1)$
values can change	Fenwick or segment tree	$O(\log n)$

When the data never changes, preprocessing can replace a dynamic data structure. Sparse tables answer idempotent range queries in $O(1)$ after $O(n \log n)$ preprocessing.

This structure is called a **sparse table**. It is a table of precomputed answers for every power-of-two range length and every valid starting position. The word “sparse” is historical terminology; the important idea is the power-of-two decomposition described below.

An operation is idempotent when:

```
op(x, x) = x
```

Minimum and gcd are idempotent. Sum is not.

Store:

```
st[k][i] = answer for range [i, i + 2^k - 1]
```

The table contains one row for each block length. For [5, 1, 4, 7, 2, 9]:

```
k = 0, length 1: [5] [1] [4] [7] [2] [9]
k = 1, length 2: [5,1] [1,4] [4,7] [7,2] [2,9]
k = 2, length 4: [5,1,4,7] [1,4,7,2] [4,7,2,9]
```

Blocks on the same row overlap because a block is stored for every possible starting position. That is what lets a query begin at any index.

Build by joining two half-length blocks:

```
st[k][i] = op(st[k-1][i], st[k-1][i + 2^(k-1)])
```

The two smaller blocks are adjacent:

```
|----- length 2^k -----|
|---- length 2^(k-1) ---|---- length 2^(k-1) ---|
start i                      i + 2^(k-1)
```

Level zero is the original array. If level $k-1$ is correct, combining those two halves makes level k correct. This induction is the build's correctness argument. There are $O(\log n)$ levels and at most n entries per level, so the preprocessing time and memory are both $O(n \log n)$.

A query $[l, r]$ has length len . Let $k = \text{floor}(\log_2(len))$. Two overlapping blocks of length 2^k cover the range:

```
[l, l+2^k-1] and [r-2^k+1, r]
```

Why do they cover everything? Since 2^k is the largest power that fits, it is greater than half of len (unless it equals len). A block longer than half placed at each end cannot leave a gap in the middle.

Overlap is fine because the operation is idempotent.

2 Sparse table worked query

Array:

```
index: 1  2  3  4  5  6
value: 5  1  4  7  2  9
```

Query minimum on [2,5]. The length is 4, so $k = 2$ and one block of length 4 exactly covers the range:

```
st[2][2] = min([2,5])
```

Query minimum on [2,6]. The length is 5, so $k = 2$, block length 4. Use:

```
[2,5] and [3,6]
```

They overlap, but $\min(x,x)=x$, so overlap does not change the answer. For sums this would double-count the overlap, which is why sparse tables do not answer ordinary range sums this way.

2.1 Complete sparse table

The implementation uses 0-based positions internally. An integer log table avoids floating-point $\log_2$ errors near exact powers of two.

```
class SparseTableMin {
    vector<vector<long long>> st;
    vector<int> lg;
public:
    explicit SparseTableMin(const vector<long long>& a) {
        int n = static_cast<int>(a.size());
        lg.assign(n + 1, 0);
        for (int len = 2; len <= n; ++len)
            lg[len] = lg[len / 2] + 1;
        int levels = lg[n] + 1;
        st.assign(levels, vector<long long>(n));
        st[0] = a;
        for (int k = 1; k < levels; ++k) {
            int half = 1 << (k - 1), length = 1 << k;
            for (int i = 0; i + length <= n; ++i)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + half]);
        }
    }
}
```

```

// Inclusive, 0-based range [left, right].
long long query(int left, int right) const {
    int length = right - left + 1;
    int k = lg[length], block = 1 << k;
    return min(st[k][left], st[k][right - block + 1]);
}
};

```

```

class SparseTableMin:
    def __init__(self, values):
        n = len(values)
        self.log = [0] * (n + 1)
        for length in range(2, n + 1):
            self.log[length] = self.log[length // 2] + 1
        levels = self.log[n] + 1
        self.table = [[0] * n for _ in range(levels)]
        self.table[0] = values[:]
        for k in range(1, levels):
            half, length = 1 << (k - 1), 1 << k
            for i in range(n - length + 1):
                self.table[k][i] = min(self.table[k - 1][i],
                                       self.table[k - 1][i + half])

# Inclusive, 0-based range [left, right].
def query(self, left, right):
    length = right - left + 1
    k = self.log[length]
    block = 1 << k
    return min(self.table[k][left],
               self.table[k][right - block + 1])

```

For range GCD, replace min by gcd; the structure is unchanged.

3 Binary lifting: precompute long parent jumps

3.1 The problem with following parents one by one

Suppose every node stores only its direct parent. To find the 6th ancestor of a node, we could follow six parent pointers. That is correct, but a query can take $O(n)$ time on a long chain.

Binary lifting adds shortcuts. From every node, we precompute where we arrive after 1 edge, 2 edges, 4 edges, 8 edges, and so on.

Consider this tree:

```
1
|
2
|
3
|
4
|
5
|
6
|
7
```

For node 7, useful shortcuts are:

```
1 edge upward:  parent(7) = 6
2 edges upward:  7 -> 6 -> 5
4 edges upward:  7 -> 6 -> 5 -> 4 -> 3
```

We do not store a special jump of length 3, 5, or 6. Powers of two are enough because every distance can be written as a sum of powers of two.

3.2 What one table entry means

Define:

```
up[k][v] = node reached after exactly  $2^k$  upward edges
```

The exponent k is a table level, not the number of edges:

Level k	Jump length 2^k	Meaning
0	1	parent
1	2	grandparent
2	4	fourth ancestor
3	8	eighth ancestor

This distinction is essential: $\text{up}[2][v]$ means four edges, not two.

3.3 Building longer jumps from shorter jumps

The first row is known directly:

```
up[0][v] = parent[v]
```

To move four edges from v , first move two edges to an intermediate node, then move another two edges from there:

```
v ---- 2 edges ----> middle ---- 2 edges ----> result
    up[1][v]           up[1][middle]
```

In general:

```
middle = up[k-1][v]
up[k][v] = up[k-1][middle]
```

or, written in one line:

```
up[k][v] = up[k-1][ up[k-1][v] ]
```

Both smaller jumps have length $2^{(k-1)}$, so together they have length 2^k . This is the reason for the recurrence.

3.4 A complete table on a small tree

Use the tree with parents 2→1, 3→1, 4→2, 5→2, 6→4, and 7→6. Node 0 means that the requested ancestor does not exist.

v	1	2	3	4	5	6	7	
$\text{up}[0][v]$	0	1	1	2	2	4	6	(1 edge)
$\text{up}[1][v]$	0	0	0	1	1	2	4	(2 edges)
$\text{up}[2][v]$	0	0	0	0	0	0	1	(4 edges)

For example:

```
up[1][7] = up[0][ up[0][7] ]  
          = up[0][6]  
          = 4
```

Starting at 7 and moving two edges indeed gives 7 -> 6 -> 4.

Node 0 is a sentinel. We leave every `up[k][0]` equal to 0. Consequently, once a jump goes above the root, later table accesses remain safe.

4 Using the table to answer a query

Suppose we want the 6th ancestor of node 7. Write the distance in binary:

```
6 = 4 + 2
6 = 110 in binary
```

The set bits say which shortcuts to use:

```
start at 7
use the 2-edge jump: up[1][7] = 4
use the 4-edge jump: up[2][4] = 0
result: no such ancestor
```

This matches the literal path: node 7 has only five ancestors 6, 4, 2, 1, then no node.

For the 3rd ancestor:

```
3 = 2 + 1
7 --2 edges--> 4 --1 edge--> 2
```

so the answer is node 2. For distance zero, no bit is set, no jump is made, and the answer is the starting node itself.

4.1 Why the query is logarithmic

A distance at most n has only $O(\log n)$ binary digits. The query checks each digit once and performs at most one constant-time table lookup per digit. The query therefore costs $O(\log n)$ rather than $O(\text{distance})$.

5 Complete implementation

The constructor builds all shortcuts. The query then consumes the bits of the distance from least significant to most significant.

```
class BinaryLifting {
    int levels;
    vector<vector<int>> up;

public:
    // parent is 1-based. parent[root] must be 0.
    explicit BinaryLifting(const vector<int>& parent) {
        int n = static_cast<int>(parent.size()) - 1;
        levels = 1;
        while ((1LL << levels) <= n)
            ++levels;

        up.assign(levels, vector<int>(n + 1, 0));
        up[0] = parent;

        for (int k = 1; k < levels; ++k) {
            for (int node = 1; node <= n; ++node) {
                int middle = up[k - 1][node];
                up[k][node] = up[k - 1][middle];
            }
        }

        int kth_ancestor(int node, long long distance) const {
            // Consume the distance one binary digit at a time.
            for (int k = 0; k < levels && node != 0; ++k) {
                if (distance & 1LL)
                    node = up[k][node];
                distance >>= 1;
            }

            // Remaining bits need a level that was not stored.
            if (distance != 0)
                return -1;
            return node == 0 ? -1 : node;
        }
    };
};
```

```
class BinaryLifting:
    # parent is 1-based. parent[root] must be 0.
```

```

def __init__(self, parent):
    n = len(parent) - 1
    self.levels = max(1, n.bit_length())
    self.up = [[0] * (n + 1) for _ in range(self.levels)]
    self.up[0] = parent[:]

    for k in range(1, self.levels):
        for node in range(1, n + 1):
            middle = self.up[k - 1][node]
            self.up[k][node] = self.up[k - 1][middle]

def kth_ancestor(self, node, distance):
    for k in range(self.levels):
        if node == 0:
            break
        if distance & 1:
            node = self.up[k][node]
        distance >>= 1

    # Remaining bits require a jump level that was not stored.
    if distance != 0:
        return -1
    return -1 if node == 0 else node

```

Building the table costs $O(n \log n)$ time and memory. Each ancestor query costs $O(\log n)$.

6 Binary lifting correctness summary

The algorithm rests on two facts:

1. $\text{up}[k][v]$ is exactly 2^k edges above v . The recurrence proves this by joining two correct half-jumps.
2. The selected powers of two sum to exactly the requested distance because they are precisely the set bits of its binary representation.

Useful boundary tests are $(v, 0)$, $(\text{root}, 1)$, a distance equal to a node's depth, and a distance one greater than its depth.

7 Recognition

Static range min/gcd/max? Sparse table.
Static range sum? Prefix sums instead.
Repeated ancestor jumps? Binary lifting.
Need updates? Segment tree or Fenwick, not sparse table.

8 Local exercises

- A. Static Range Minimum: Given an immutable array, answer minimum queries on subarrays.
- B. Static Range GCD: Given an immutable positive array, answer gcd queries on subarrays.
- C. Kth Ancestor: In a rooted tree with root 1, answer queries asking for the k -th ancestor of a node, or -1 if it does not exist.

9 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution \  
python3 sections/38_static_queries_binary_lifting/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Exercises

Use A and B to check the idempotence requirement of sparse-table queries, and C to define `up[j][v]` before lifting bits of `k`. Test a length-one range and an ancestor above the root.

10 Static means a different tradeoff

The signal is not merely “many queries”; it is “many queries and no updates.” For a static range minimum or gcd, spend $O(n \log n)$ memory and preprocessing to make every query $O(1)$. A sparse table’s invariant is:

```
st[k][i] is the aggregate of exactly [i, i+2k-1].
```

The build recurrence joins two adjacent valid half-blocks, so induction proves the invariant. A query uses two blocks of length $2^{\lfloor \log_2(\text{len}) \rfloor}$; their union covers the range. Their overlap is legal only for idempotent operations: $\min(x, x) = x$ and $\gcd(x, x) = x$, but $\text{sum}(x, x)$ double-counts.

10.1 Second sparse-table trace

For $[12, 18, 6, 10, 15]$, query gcd $[2, 5]$ has length four, so use one length-4 block and answer 1? No: $\gcd(18, 6, 10, 15) = 1$, which is exactly $\text{st}[2][2]$ in 1-based notation. For a length-five query, use two length-four overlapping blocks; applying gcd to both remains correct because duplicate values do not change a gcd.

11 Notes, mistakes, and formative checks

Precompute integer logs with `lg[i]=lg[i//2]+1`, not floating `log2`. In C++, use `1LL<<k` when comparing a potentially large ancestor distance. In Python, stop the bit loop when the node becomes zero or the distance becomes zero.

Mistake	Test
sparse table for sums using overlap	sum query of length 3
wrong right block start	range of non-power-of-two length
forget 1-based to 0-based conversion	singleton range
omit sentinel root parent	lift above the root

1. Why can min use overlapping blocks? Answer: repeating any candidate leaves the minimum unchanged. 2. Why cannot sum? Answer: overlap contributes twice.
2. What is the 0th ancestor? Answer: the node itself.

12 Submission workflow

For sparse tables, compare each query to a direct scan on small arrays. For lifting, climb parents literally for small distances. Test length-one ranges, non-power-of-two lengths, root queries, zero distance, and distance above depth.

```
CP_TARGET=solution \  
python3 sections/38_static_queries_binary_lifting/check.py
```